

Assignment 5 (Week 5): Queues, Circular Buffers, Deques, and Priority

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 17, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 5’s Notes chapter alone. Notation matches the lecture exactly: **enqueue** adds at **rear**, **dequeue** removes from **front**, **front** peeks without removing, indices are 0-based on $Q[0..n-1]$ with capacity n , circular advance is $(\cdot+1) \bmod n$, deque operations are **enqueueFront**/**enqueueRear**/**dequeueFront**/**dequeueRear**, and priority removal is **deleteMin**. For every bound, name the operation, give the bound, and state the case (Week 2 shape).

5.1 Conceptual

Q1. A linear array queue reports “full” while free slots stand empty — overflow with free space (standard treatment; see Horowitz & Sahni Ch. 3) [1].

- (a) In one or two sentences, explain the **false-full** trap: with **rear** = $n - 1$ and **front** > 0, where exactly are the free slots, and why can the linear code never reuse them?
- (b) In one or two sentences, explain why the shift-everything-left-on-dequeue fix is dishonest pricing: what does each **dequeue** cost worst case, and which term of the queue contract (**enqueue**/**dequeue** each $O(1)$ worst case) does it break?
- (c) In one or two sentences, contrast with the Week 3 stack: why does a single moving boundary (**top**) never strand slots the way two marching indices (**front**, **rear**) do?

Why this matters: a print spooler that cries “full” with an empty half-buffer is not out of memory, it is out of arithmetic — this question pins down whose arithmetic is at fault before the fix is introduced.

Q2. With wraparound, **front** = **rear** after the first enqueue *and* when the ring is completely full — one equality, two meanings (Karumanchi Ch. 5, pp.205–223 PDF) [2].

- (a) In one or two sentences, state the ambiguity precisely: name the two distinct ring states that both satisfy **front** = **rear**, and explain why testing full as **front** = **rear** also fires on empty.
- (b) Compare the two honest fixes in a small table (one row each): the test expression, the number of items the ring can hold at capacity n , and the extra state required. Name which fix the course adopts (Lab P5) and why in one sentence.

- (c) Unit-I review: in one or two sentences, explain why writing the full/empty test down *before* the code prevents the most common queue bug on a Class Test 1 paper.

Why this matters: examiners check the test, not the fashion — this question makes you state both tests exactly, so the name alone earns the first mark and the trace earns the rest.

5.2 Numerical

Q3. Circular queue, `count` version, capacity $n = 6$. Start at `front = 4`, `rear = 2`, `count = 5` (five live items sit at indices 4, 5, 0, 1, 2). Rules: `dequeue` advances `front = (front + 1) mod n` and decrements `count`; `enqueue` advances `rear = (rear + 1) mod n` and increments `count`; full iff `count = n`.

- (a) Run, in order: `dequeue`, `enqueue(V)`, `enqueue(W)`, `dequeue`, `enqueue(X)`. Give `front`, `rear`, `count` after *every* step as a five-row table.
- (b) Show each modular computation explicitly (all five advances), and circle each step where an index wraps past $n - 1$ back to 0 (there may be fewer such steps than advances).
- (c) Is the queue full at the end? How many further enqueues (if any) can it accept before the honest full test fires?

Why this matters: the wrap steps are exactly where the linear code died — this question makes your hand execute the one-line arithmetic that refuses to die there.

Q4. Two pricings: one scan and two workloads (Karumanchi Ch. 7, pp.369–408 PDF; Sedgewick Sec. 2.4, p. 308) [2, 3].

- (a) An unordered array priority queue holds T:6, U:2, V:9, W:4 (smaller number = more urgent). Trace `deleteMin`: which job is returned, how many priorities are scanned, and what is the worst-case cost? Name the operation, give the bound, state the case.
- (b) Workload A: 800 inserts then 4 removals. Workload B: 4 inserts then 600 removals. For each workload, write the total asymptotic bill under *both* designs (unordered: $O(1)$ insert, $O(n)$ `deleteMin`; ordered: $O(n)$ insert, $O(1)$ `deleteMin`), pick the cheaper design, and state the deck's workload rule in one sentence.

Why this matters: there is no free lunch with arrays, so you price the workload, not the structure — this question puts both bills for both workloads on the same page.

5.3 Design

Q5. A deque on a capacity-6 ring supports all four operations at $O(1)$ worst case each; `front` may step backwards as `front = (front - 1 + n) mod n` (Karumanchi Ch. 5, pp.205–223 PDF) [2]. An **input-restricted** deque inserts at one end only and removes at either; an **output-restricted** deque inserts at either end and removes at one end only.

- (a) Starting empty, run: `enqueueRear(M)`, `enqueueFront(N)`, `enqueueRear(P)`, `dequeueFront`, `enqueueFront(Q)`, `dequeueRear`. Give the live contents (front to rear) after *every* step.
- (b) Classify the sequence: does it stay inside the input-restricted variant, the output-restricted variant, both, or neither? Justify in one or two sentences by naming which end(s) each operation uses.

- (c) Design a four-operation sequence (using at least one `enqueueFront` and at least one removal from each end across the whole sequence is *not* required — choose freely) that is legal in an output-restricted deque but illegal in an input-restricted one. List the operations, give the final contents, and state in one sentence which restriction bites and where.

Why this matters: the name earns the first mark only if you can tie each operation to the end it touches — this question forces that tie on a sequence the lecture never worked.

Q6. Design review: a print spooler buffers jobs in a capacity-5 array. The arrival pattern is fixed: `enqueue(a)`, `enqueue(b)`, `enqueue(c)`, `dequeue`, `dequeue`, `enqueue(d)`, `enqueue(e)`, `enqueue(f)`.

- (a) Run the pattern on the *linear* implementation (start `front = 0`, `rear = -1`; `rear = rear + 1` with no wraparound). Give `front/rear` after every step, pinpoint the exact step where the false full fires, and state how many live jobs and how many free-but-unreachable slots exist at that moment.
- (b) Run the *same* pattern from empty on the circular `count` version. Give `front/rear/count` after every step, show the wraparound computation, and state whether the pattern completes and whether the buffer is full at the end.
- (c) Had the spooler used the sacrificed-slot fix at $n = 5$ instead: how many jobs could it hold at most, what is the exact full test, and in one or two sentences, when would you choose it over the `count` version (name the resource each fix spends)?

Why this matters: this is the week’s whole arc as one shipping decision — the linear buffer fails the acceptance test, the ring passes it, and the capacity ledger tells you what the alternative fix would have cost.

References

- [1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksources, 2nd edition, 2008.
- [2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [3] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.